CO2 – DEEP FEED-FORWARD AND CONVOLUTIONAL NETWORK DESIGN

Assessment: Smart Agriculture and Crop Disease Detection

Nature: Individual
Duration: 1 Week
Total Marks: 50

TASK 1 – Deep Feed-Forward Network Architecture

a) Design the network architecture

A feed-forward neural network is used to predict crop yield using four input features:

1. Rainfall
2. Temperature
3. Soil Moisture
4. Fertilizer Quantity

Given:

- Input Layer = 4 neurons
- Hidden Layer 1 = 8 neurons
- Hidden Layer 2 = 4 neurons
- Output Layer = 1 neuron

Network Architecture:

Input Layer (4) → Hidden Layer 1 (8) → Hidden Layer 2 (4) → Output Layer (1)

Total neurons = 4 + 8 + 4 + 1 = 17 neurons.

b) Dimensions of weight matrices and bias vectors

Input Layer to Hidden Layer 1:

$W1 = 8 \times 4$

$b1 = 8 \times 1$

Hidden Layer 1 to Hidden Layer 2:

W2 = 4 × 8

b2 = 4 × 1

Hidden Layer 2 to Output Layer:

W3 = 1 × 4

b3 = 1 × 1

Therefore:

- W1 = (8, 4)
- W2 = (4, 8)
- W3 = (1, 4)

c) How inputs flow through the network

The network uses forward propagation to calculate crop yield.

Step 1: Input Layer

X = [Rainfall, Temperature, Soil Moisture, Fertilizer]

Step 2: First Hidden Layer

Z1 = W1X + b1

A1 = ReLU(Z1)

Step 3: Second Hidden Layer

Z2 = W2A1 + b2

A2 = ReLU(Z2)

Step 4: Output Layer

Z3 = W3A2 + b3

The final output is the predicted crop yield.

A linear activation function is suitable for the output layer because crop yield is a continuous value.

Conclusion: A feed-forward network learns relationships between rainfall, temperature, soil moisture and fertilizer quantity to predict crop yield.

TASK 2 – Forward Propagation and Activation Functions

a) Calculate the weighted sum

Given:

Inputs = (0.6, 0.8, 0.5)

Weights = (0.4, 0.3, 0.2)

Bias = 0.1

Formula:

$Z = (X1 \times W1) + (X2 \times W2) + (X3 \times W3) + Bias$

Substituting:

$Z = (0.6 \times 0.4) + (0.8 \times 0.3) + (0.5 \times 0.2) + 0.1$

$Z = 0.24 + 0.24 + 0.10 + 0.10$

$Z = 0.68$

b) Apply ReLU activation function

Formula:

$ReLU(Z) = max(0, Z)$

$ReLU(0.68) = max(0, 0.68)$

Output = 0.68

Interpretation:

Since the weighted sum is positive, ReLU returns 0.68.

ReLU converts negative values into zero and keeps positive values unchanged.

Conclusion: Activation functions introduce non-linearity and help neural networks learn complex relationships.

TASK 3 – Gradient-Based Learning and Loss Calculation

Given:

Actual crop yield (y) = 50 units

Predicted crop yield (ŷ) = 42 units

a) Why is a loss function required?

A loss function measures the difference between actual and predicted outputs.

For regression problems, Mean Squared Error (MSE) is commonly used.

Formula:

$MSE = (1/n) \Sigma(y - \hat{y})^2$

It calculates the average squared difference between actual and predicted values.

b) Calculate squared error loss

Given:

$y = 50$

$\hat{y} = 42$

Formula:

$L = (y - \hat{y})^2$

Substituting:

$L = (50 - 42)^2$

$L = 8^2$

$L = 64$

c) Interpretation of loss value

The loss value is 64.

It indicates that the squared difference between actual crop yield and predicted crop yield is 64.

A lower loss indicates better prediction performance, while a higher loss indicates a larger prediction error.

d) How gradient-based learning updates network parameters

Gradient-based learning minimizes the loss by updating weights and biases.

Steps:

1. Forward propagation calculates the predicted crop yield.
2. The loss function calculates prediction error.
3. Backpropagation calculates gradients of the loss with respect to weights and biases.
4. Gradient descent updates the parameters.
5. The process repeats until the loss becomes sufficiently small.

Weight update formula:

$W_{new} = W_{old} - \eta \times (\partial L/\partial W)$

Where:

- W = Weight
- η = Learning rate
- $\partial L/\partial W$ = Gradient of loss.

Conclusion: Gradient-based learning reduces prediction errors by continuously updating network parameters.

TASK 4 – Gradient Descent and Weight Update

Given:

Initial weight (W) = 0.5

Gradient = 0.8

Learning rate (η) = 0.1

a) Calculate the updated weight

Formula:

$W_{new} = W_{old} - \eta \times Gradient$

Substituting:

$W_{new} = 0.5 - (0.1 \times 0.8)$

$W_{new} = 0.5 - 0.08$

$W_{new} = 0.42$

b) Why is the update in the opposite direction to the gradient?

The gradient indicates the direction of the greatest increase in loss.

To minimize the loss, gradient descent moves in the opposite direction.

Therefore, the gradient is subtracted from the current weight.

c) Effects of extremely small or large learning rates

Small Learning Rate:

1. Weight updates become very small.
2. Training takes more time.
3. Convergence becomes slow.
4. More iterations are required.

Large Learning Rate:

1. Weight updates become very large.
2. The model may skip the minimum loss point.
3. Training may become unstable.
4. Loss may fluctuate or increase.

Conclusion: A suitable learning rate is important for stable and efficient neural network training.

TASK 5 – Back-Propagation Through Multiple Layers

a) How backpropagation determines the contribution of weights to prediction error

Backpropagation calculates gradients of the loss function with respect to network weights and biases.

Steps:

1. Forward propagation calculates predicted crop yield.
2. The loss function calculates the prediction error.
3. Backpropagation calculates the error at the output layer.
4. The error is propagated backwards through hidden layers.
5. Gradients are calculated for each weight and bias.

This identifies how each parameter contributes to prediction error.

b) Role of chain rule

The chain rule calculates gradients across multiple layers.

For example:

$\partial L/\partial W = (\partial L/\partial Z) \times (\partial Z/\partial W)$

For multiple layers, the chain rule multiplies the derivatives of successive operations.

It allows gradients to flow from the output layer to earlier layers.

c) How gradients update weights

After calculating gradients, gradient descent updates each weight.

Formula:

$W_{new} = W_{old} - \eta \times (\partial L/\partial W)$

Where:

- W_{old} = Previous weight
- η = Learning rate
- $\partial L/\partial W$ = Gradient.

The process repeats until the network minimizes the loss.

Conclusion: Backpropagation calculates gradients using the chain rule and helps neural networks learn by updating weights and biases.

TASK 6 – Activation Functions and Vanishing/Exploding Gradients

a) How activation functions contribute to vanishing gradients

Vanishing gradient occurs when gradients become extremely small while moving backwards through a deep neural network.

As a result, earlier layers learn very slowly or stop learning.

Sigmoid and tanh can produce very small derivatives in certain regions.

When these small derivatives are multiplied across many layers, gradients become extremely small.

This causes the vanishing gradient problem.

b) Comparison of sigmoid, tanh and ReLU

1. Sigmoid Function

Formula:

$f(x) = 1 / (1 + e^{-x})$

- Output range: 0 to 1.
- Commonly used in binary classification output layers.
- Can cause vanishing gradients.
- Not zero-centred.

2. Tanh Function

Formula:

$f(x) = (e^{x} - e^{-x}) / (e^{x} + e^{-x})$

- Output range: -1 to +1.
- Zero-centred.
- Can also cause vanishing gradients.
- Often provides better optimization behaviour than sigmoid in hidden layers.

3. ReLU Function

Formula:

$f(x) = \max(0, x)$

- Output range: 0 to infinity.
- Computationally simple.
- Helps reduce vanishing gradients for positive inputs.
- May suffer from the dying ReLU problem when neurons continuously output zero.

c) Effect of activation-function choice

1. Activation functions introduce non-linearity.
2. They help networks learn complex patterns.
3. ReLU is commonly preferred for hidden layers.
4. Sigmoid is commonly used for binary classification outputs.
5. Softmax is commonly used for multiclass classification outputs.

Conclusion: Choosing suitable activation functions improves gradient flow and makes deep network training more effective.

TASK 7 – Back-Propagation and Optimization Strategy

a) Effect of learning rate on optimization

Learning rate controls the size of weight updates during training.

- Very small learning rate: Training becomes slow.
- Suitable learning rate: Loss decreases steadily.
- Very large learning rate: Loss may fluctuate or diverge.

Therefore, selecting a suitable learning rate improves convergence.

b) How backpropagation and gradient descent work together

One training iteration consists of the following steps:

Step 1: Forward Propagation

Input data passes through the network to produce predictions.

Step 2: Loss Calculation

The loss function measures the difference between actual and predicted values.

Step 3: Backpropagation

Gradients are calculated using the chain rule.

Step 4: Gradient Descent

Weights and biases are updated using the calculated gradients.

Step 5: Repeat

The process continues until the loss decreases.

c) Changes to improve convergence

Experiment 1: Training loss decreases slowly.

1. Increase the learning rate slightly.
2. Use an adaptive optimizer such as Adam.
3. Apply suitable weight initialization.
4. Normalize input features.

Experiment 2: Training loss decreases quickly but fluctuates considerably.

1. Reduce the learning rate.
2. Use momentum.
3. Apply a learning-rate scheduler.
4. Use mini-batch training with a suitable batch size.

d) Importance of monitoring the loss curve

A loss curve shows how training and validation losses change over time.

It helps identify:

1. Whether the model is learning properly.
2. Whether the learning rate is suitable.
3. Whether training is unstable.
4. Whether the model is overfitting.
5. When training should be stopped using early stopping.

Conclusion: Backpropagation calculates gradients, while optimization algorithms use them to update weights and minimize loss.

TASK 8 – Regularization and Overfitting Control

Given:

Training Accuracy = 99%

Validation Accuracy = 78%

a) Identify the learning problem

The given model suffers from overfitting.

Reasons:

1. Training accuracy is very high at 99%.
2. Validation accuracy is much lower at 78%.
3. The model learns training data too closely.
4. It fails to generalize well to unseen data.

b) Explain L1 and L2 regularization

Regularization is used to reduce overfitting by controlling model complexity.

1. L1 Regularization

L1 adds the absolute values of weights to the loss function.

Formula:

$L_{total} = L_{original} + \lambda \Sigma |W|$

Advantages:

1. Encourages some weights to become exactly zero.
2. Helps perform feature selection.
3. Produces a simpler model.

2. L2 Regularization

L2 adds the squared values of weights to the loss function.

Formula:

$L_{total} = L_{original} + \lambda \Sigma W^2$

Advantages:

1. Penalizes large weights.
2. Reduces model complexity.
3. Helps improve generalization.
4. Prevents weights from becoming excessively large.

c) Regularized loss function using L2 penalty

Formula:

$L_{total} = L_{original} + \lambda \Sigma W^2$

Where:

- L_{total} = Total loss
- $L_{original}$ = Original prediction loss
- λ = Regularization parameter
- W = Model weights.

The regularization parameter controls the strength of the penalty.

- Small λ: Weak regularization.
- Large λ: Strong regularization.

A suitable value of λ helps control overfitting.

d) How dropout improves generalization

Dropout randomly disables some neurons during training.

Advantages:

1. Prevents neurons from depending too much on each other.
2. Reduces overfitting.
3. Encourages the network to learn independent features.
4. Improves performance on unseen data.

For example, with a dropout rate of 0.5, approximately 50% of the selected layer's neurons are temporarily disabled during each training step.

Conclusion: L1, L2 and dropout help control model complexity and improve generalization.

TASK 9 – Convolution Operation and Feature Extraction

a) Why convolutional layers are suitable for image data

Convolutional Neural Networks (CNNs) are suitable for images because they automatically identify visual features.

Advantages:

1. Detect edges and shapes.
2. Identify spots and disease patterns.
3. Preserve spatial relationships between pixels.
4. Reduce parameters through weight sharing.
5. Learn important image features automatically.

b) Demonstrate one convolution operation

Consider the following 3 × 3 image matrix:

Image Matrix:

[1 2 0]

[3 1 2]

[0 1 3]

Consider a 2 × 2 filter:

Filter:

[1 0]

[0 -1]

Multiply corresponding values and add them.

First operation:

= (1 × 1) + (2 × 0) + (3 × 0) + (1 × -1)

= 1 + 0 + 0 - 1

= 0

Second operation:

= (2 × 1) + (0 × 0) + (1 × 0) + (2 × -1)

= 2 + 0 + 0 - 2

= 0

Third operation:

= (3 × 1) + (1 × 0) + (0 × 0) + (1 × -1)

= 3 + 0 + 0 - 1

= 2

Fourth operation:

= (1 × 1) + (2 × 0) + (1 × 0) + (3 × -1)

= 1 + 0 + 0 - 3

= -2

Final Feature Map:

[0 0]

[2 -2]

Interpretation:

The filter extracts specific patterns from the image by performing multiplication and addition.

In actual CNNs, filters learn useful patterns such as edges, textures and disease spots during training.

c) Roles of filters, stride and padding

Filters:

Filters are small matrices that identify specific patterns such as edges, textures and shapes.

Stride:

Stride specifies how many pixels the filter moves at each step.

- Stride 1: Moves one pixel at a time.
- Stride 2: Moves two pixels at a time.

A larger stride reduces the output feature-map dimensions.

Padding:

Padding adds extra pixels, usually zeros, around the image boundary.

Advantages:

1. Preserves spatial dimensions when required.
2. Helps detect features near image boundaries.
3. Controls output feature-map size.

Conclusion: Convolution extracts important visual features from crop-leaf images and helps identify disease patterns.

TASK 10 – CNN Architecture and Pooling

Given CNN Architecture:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected Layer → Output

a) Explain the function of each component

1. Input Image

The input layer receives crop-leaf images.

Example: 32 × 32 × 3 RGB image.

2. Convolution Layer

- Applies filters to the input image.
- Extracts important features such as edges, spots and textures.
- Produces feature maps.

3. ReLU Activation

Formula:

ReLU(x) = max(0, x)

- Converts negative values into zero.
- Keeps positive values unchanged.
- Introduces non-linearity.

4. Pooling Layer

- Reduces feature-map dimensions.
- Reduces computational requirements.
- Retains important features.

5. Fully Connected Layer

- Combines extracted features.
- Performs classification based on learned patterns.

6. Output Layer

Produces the final classification.

Example:

- Healthy Leaf
- Diseased Leaf

For binary classification, sigmoid activation can be used in the output layer.

b) How pooling reduces feature-map dimensions

Pooling reduces the spatial size of feature maps.

Example:

Original Feature Map (4 × 4):

[1 3 2 4]

[5 6 1 2]

[7 2 8 3]

[4 1 5 9]

Using 2 × 2 Max Pooling with stride 2:

First block:

[1 3]

[5 6]

Maximum = 6

Second block:

[2 4]

[1 2]

Maximum = 4

Third block:

[7 2]

[4 1]

Maximum = 7

Fourth block:

[8 3]

[5 9]

Maximum = 9

Final Pooled Feature Map:

[6 4]

[7 9]

Therefore:

Original dimensions = 4 × 4

After pooling = 2 × 2

The feature-map size is reduced from 16 values to 4 values.

c) Effect of filter size, stride and number of filters

Filter Size:

- Small filters such as 3 × 3 detect local features.
- Large filters cover larger image regions.
- Filter size affects the amount of spatial information captured.

Stride:

- Controls how far the filter moves.
- Larger stride produces smaller feature maps.
- Smaller stride preserves more spatial information.

Number of Filters:

- More filters extract more types of features.
- Fewer filters reduce computational requirements.
- The number of filters determines the output depth of the feature map.

d) How multiple convolutional layers learn complex features

CNNs learn features in a hierarchical manner.

1. First convolutional layer: Detects simple edges and lines.
2. Second convolutional layer: Detects textures, spots and shapes.
3. Deeper convolutional layers: Detect complex disease patterns and leaf structures.
4. Fully connected layer: Combines extracted features for classification.

Conclusion: CNNs use convolution, activation and pooling layers to automatically extract important features and classify crop diseases.

TASK 11 – Comparative Evaluation of Feed-Forward and CNN Models

a) Why Model A is appropriate for structured numerical data

Model A is a feed-forward neural network that uses numerical features such as:

1. Rainfall
2. Temperature
3. Soil moisture
4. Fertilizer quantity

Feed-forward networks are suitable for structured numerical data because they learn relationships between input features and output values.

They are useful for predicting crop yield.

b) Why Model B is suitable for images

Model B is a Convolutional Neural Network (CNN).

CNNs are suitable for image data because they automatically extract visual features.

They can identify:

1. Leaf spots.
2. Colour changes.
3. Edges and shapes.
4. Disease patterns.
5. Texture differences.

Therefore, CNNs are suitable for crop-leaf disease classification.

c) Comparison between feed-forward networks and CNNs

1. Input Representation

Feed-Forward Network:

Uses numerical feature vectors such as rainfall and temperature.

CNN:

Uses image matrices containing pixel values.

2. Feature Extraction

Feed-Forward Network:

Learns relationships between numerical input features through hidden layers.

CNN:

Automatically extracts image features using convolutional filters.

3. Parameter Organization

Feed-Forward Network:

Uses fully connected weight matrices between consecutive layers.

CNN:

Uses shared convolutional filters, reducing the number of parameters compared with fully connected image processing.

4. Computational Requirements

Feed-Forward Network:

Generally requires fewer computational resources for small structured datasets.

CNN:

Requires more computation for image processing because it performs convolution operations over image regions.

d) Recommended model for each task

For Crop Yield Prediction:

Recommended Model: Feed-Forward Neural Network.

Reason:

It can learn relationships between rainfall, temperature, soil moisture and fertilizer quantity to predict continuous crop-yield values.

For Crop Disease Classification:

Recommended Model: CNN.

Reason:

It can automatically identify visual disease patterns from crop-leaf images and classify them as healthy or diseased.

Conclusion: Feed-forward networks are suitable for numerical prediction, whereas CNNs are suitable for image classification.

TASK 12 – DESIGN CHALLENGE: OPTIMIZE AND REDESIGN THE DEEP LEARNING SYSTEM

Changed Requirement:

The agricultural centre initially developed a feed-forward network using 10,000 crop records. Now it must process:

- 500,000 numerical crop records.
- 100,000 crop-leaf images.

The existing model has high training accuracy, low validation performance and long training time.

The system should be redesigned using separate architectures for numerical data and image data.

A. Impact Analysis (2 Marks)

a) Effect of larger datasets and additional leaf images

Increasing the dataset size affects computational and architectural requirements.

1. Numerical Dataset

The number of crop records increases from 10,000 to 500,000.

This requires:

- More memory and storage.
- More computational operations.
- Efficient data loading.
- Suitable batch sizes.
- Efficient optimization algorithms.

2. Addition of Leaf Images

The system must now process 100,000 leaf images.

Images contain large numbers of pixel values, requiring more computation.

CNN architecture is suitable because it extracts visual features using convolutional filters.

3. Training Time

A larger dataset increases training time.

GPU acceleration, mini-batch training and efficient architectures can reduce training time.

4. Model Architecture

Separate architectures are required for different data types.

- Feed-forward network for numerical crop data.
- CNN for crop-leaf images.

b) Limitations of applying feed-forward architecture directly to image data

1. Images contain spatial relationships between neighbouring pixels.
2. Fully connected networks flatten images into long vectors.
3. Spatial information may be lost.
4. Fully connected networks require a large number of parameters.
5. Training becomes computationally expensive.
6. They are less efficient at extracting local image features.

Conclusion: A CNN is more suitable for image processing because it preserves spatial information and automatically extracts visual features.

B. Evaluation of the Existing Design

a) Evaluate the existing model

The existing model has high training accuracy but low validation accuracy.

This indicates overfitting.

Reasons:

1. The model learns training examples too closely.
2. It may memorize noise and unnecessary patterns.
3. It performs poorly on unseen crop records.
4. It does not generalize effectively.

To improve the model:

- Apply dropout.
- Use L2 regularization.

- Increase training data diversity.
- Apply early stopping.
- Use data augmentation for images.
- Perform proper validation.

b) Is increasing network depth and neuron count the best solution?

No. Simply increasing the number of layers and neurons is not always the best solution.

Reasons:

1. Increasing model complexity may increase overfitting.
2. More neurons increase the number of parameters.
3. Training time and memory requirements increase.
4. The model may learn unnecessary noise.
5. Validation performance may decrease.

Instead, the model should use a suitable architecture, regularization and optimization techniques.

Conclusion: The existing model should be improved by selecting suitable architectures and controlling overfitting rather than simply increasing its complexity.

C. Adaptation or Redesign

a) Proposed feed-forward network and CNN

The agricultural centre should use two separate deep learning models.

1. Feed-Forward Network for Numerical Crop Data

Input Features:

- Rainfall
- Temperature
- Soil Moisture
- Fertilizer Quantity

Proposed Architecture:

Input Layer (4 neurons)

↓

Hidden Layer 1 (64 neurons, ReLU)

↓

Hidden Layer 2 (32 neurons, ReLU)

↓

Hidden Layer 3 (16 neurons, ReLU)

↓

Output Layer (1 neuron, Linear)

Output:

Predicted Crop Yield.

A regression loss function such as Mean Squared Error can be used.

2. CNN for Crop Disease Classification

Input:

128 × 128 × 3 RGB crop-leaf image.

Proposed Architecture:

Input Image

↓

Convolution Layer (32 filters, 3 × 3)

↓

ReLU

↓

Max Pooling (2 × 2)

↓

Convolution Layer (64 filters, 3 × 3)

↓

ReLU

↓

Max Pooling (2 × 2)

↓

Convolution Layer (128 filters, 3 × 3)

↓

ReLU

↓

Global Average Pooling

↓

Dropout

↓

Output Layer (Sigmoid)

Output:

- Healthy Leaf
- Diseased Leaf

Binary Cross-Entropy can be used as the loss function for binary classification.

b) Recommended optimization and regularization methods

1. Adam Optimizer

Adam combines momentum and adaptive learning rates.

Advantages:

- Faster convergence in many practical problems.
- Automatically adapts learning rates.
- Suitable for deep neural networks.

2. SGD Optimizer

Stochastic Gradient Descent updates model weights using gradients calculated from training examples or batches.

It is simple and computationally efficient.

3. Learning Rate Adjustment

A suitable learning rate improves training stability.

The learning rate can be reduced when validation loss stops improving.

4. Dropout

Randomly disables neurons during training to reduce overfitting.

5. L2 Regularization

Penalizes large weights and controls model complexity.

6. Batch Normalization

Normalizes intermediate activations within mini-batches.

It can improve training stability and convergence.

7. ReLU Activation

ReLU is suitable for hidden layers because it is computationally simple and helps reduce vanishing-gradient problems for positive activations.

Conclusion: Using a feed-forward network for numerical data and a CNN for images, together with suitable optimization and regularization, improves the overall system.

D. Evidence-Based Defence

a) Design experiments to compare the original and redesigned systems

The original and redesigned systems should be trained and evaluated using the same training and validation data splits wherever applicable.

Experiment 1: Crop Yield Prediction

Compare the original feed-forward network with the redesigned feed-forward network.

Evaluation Metrics:

1. Mean Squared Error (MSE).
2. Mean Absolute Error (MAE).
3. Root Mean Squared Error (RMSE).

4. Training Time.
5. Validation Loss.

Experiment 2: Crop Disease Classification

Train the redesigned CNN using crop-leaf images.

Evaluation Metrics:

1. Accuracy.
2. Precision.
3. Recall.
4. F1-score.
5. Confusion Matrix.
6. Training Time.

Experiment 3: Overfitting Analysis

Compare training and validation loss.

- Low training loss and low validation loss indicate good learning.
- Low training loss and high validation loss indicate overfitting.
- High training loss and high validation loss indicate underfitting.

Experiment 4: Computational Efficiency

Measure:

- Training time.
- Inference time.
- Memory usage.
- GPU utilization.

These experiments help determine whether the redesigned system provides better performance and efficiency.

b) Justification of the final design

The original system should be redesigned rather than retained without changes.

Reasons:

1. The dataset has increased from 10,000 to 500,000 numerical records.
2. The addition of 100,000 images requires a CNN architecture.
3. Feed-forward networks are suitable for numerical crop-yield prediction.
4. CNNs are suitable for extracting visual features from crop-leaf images.

5. Adam and SGD can improve optimization.
6. Dropout and L2 regularization help reduce overfitting.
7. Batch normalization improves training stability.
8. Data augmentation increases image diversity.
9. Suitable learning-rate scheduling improves convergence.
10. Separate evaluation metrics measure the performance of both tasks.

Final Conclusion:

The agricultural centre should redesign the existing system using a feed-forward neural network for crop-yield prediction and a CNN for crop-disease classification.

The redesigned system should use suitable optimizers, regularization, data augmentation and learning-rate scheduling.

This approach improves generalization, reduces overfitting and supports efficient processing of large numerical and image datasets.

ADDITIONAL DESIGN CONSIDERATIONS

1. Comparison of Optimization Algorithms

SGD (Stochastic Gradient Descent):

- Updates weights using individual training examples.
- Simple and memory-efficient.
- Updates can be noisy.
- Requires careful learning-rate tuning.

Mini-Batch SGD:

- Uses a small batch of training examples.
- More stable than single-example SGD.
- Efficient for GPU processing.
- Commonly used in deep learning.

Momentum:

- Uses previous gradients to improve weight updates.
- Helps accelerate convergence.
- Reduces oscillations during optimization.

Adam:

- Combines momentum and adaptive learning rates.
- Usually provides fast initial convergence.
- Suitable for many deep learning applications.

RMSProp:

- Adjusts learning rates using a moving average of squared gradients.
- Helps control large gradient updates.
- Useful for non-stationary optimization problems.

Recommended Optimizer:

Adam can be used initially for both networks. SGD with momentum can also be tested to compare validation performance and final convergence.

2. Proposed Learning-Rate Schedule

A learning-rate schedule gradually adjusts the learning rate during training.

Example:

Initial Learning Rate = 0.001

When validation loss stops improving, reduce the learning rate by a factor of 0.5.

Example:

- Initial learning rate = 0.001
- After first reduction = 0.0005
- After second reduction = 0.00025
- After third reduction = 0.000125

This helps the model make smaller updates as it approaches a minimum.

3. Label-Preserving Image Augmentation

Image augmentation increases training image diversity without changing the original disease label.

Recommended techniques:

1. Horizontal flipping.
2. Vertical flipping.
3. Small-angle rotation.
4. Random cropping.

5. Slight brightness adjustment.
6. Slight zooming.
7. Small contrast adjustment.

Example:

An image of a diseased leaf remains a diseased leaf after a small rotation or horizontal flip.

Important: Excessive colour changes should be avoided because leaf colour is an important feature for identifying diseases.

Augmentation should be applied only to training images, not validation or test images.

4. Comparison of CNN Architectures

LeNet:

- Simple and small CNN architecture.
- Suitable for small image datasets and basic image classification.
- Has relatively low computational requirements.
- Useful as a baseline model.

AlexNet:

- Deeper than LeNet.
- Uses multiple convolutional layers and ReLU activation.
- Requires more computational resources.
- Originally designed for larger images.

ResNet:

- Uses residual connections.
- Helps train deeper neural networks.
- Reduces degradation problems in very deep networks.
- Suitable for complex image classification.

EfficientNet:

- Uses a balanced scaling approach for network depth, width and image resolution.
- Provides good accuracy with efficient computation.
- Suitable for image classification with limited computational resources.

Recommended Architecture: EfficientNet-B0

Reasons:

1. Provides a good balance between accuracy and computational cost.
2. Suitable for transfer learning.
3. Can be adapted to crop-leaf disease classification.
4. Requires fewer computational resources than many larger CNN architectures.

For a limited-compute baseline, a small custom CNN or LeNet can also be tested. ResNet18 is another practical comparison model

CONCLUSION

Deep feed-forward networks and convolutional neural networks play important roles in modern deep learning applications.

Feed-forward networks learn relationships between structured numerical features and are suitable for crop-yield prediction.

CNNs automatically extract important visual features from crop-leaf images and are suitable for disease classification.

Forward propagation calculates predictions, while backpropagation calculates gradients to update network parameters. Gradient descent and optimization algorithms minimize prediction loss.

Activation functions introduce non-linearity, while regularization techniques such as L1, L2 and dropout help control overfitting.

Convolution and pooling operations help CNNs identify important image features while reducing computational complexity.

Therefore, combining suitable network architectures, optimization techniques, regularization and image augmentation helps develop an efficient and reliable smart agriculture deep learning system.